

# Fault Tree Analysis

## Detailed Procedure

*How the v6 pipeline constructs a hybrid fault tree, validates hypotheses through two evidence gates, cross-references commanded versus actual values, and synthesizes a verified root cause. Worked end-to-end with a 5G handover example.*

| SCOPE                 | PER              | SKILLS INVOLVED               | COMPANION             |
|-----------------------|------------------|-------------------------------|-----------------------|
| Phase 3.1 through 3.5 | Single iteration | 4 phase skills + 1 controller | v6-design-review.html |

### CONTENTS

- 01 FTA philosophy & invariants
- 02 Pipeline at a glance
- 03 Phase 3.1 – Build hybrid tree
- 04 Phase 3.2 – Gate A pivot-pruning
- 05 Phase 3.3 – Expand & Gate B
- 06 Phase 3.4 – Cross-reference
- 07 Phase 3.5 – Root cause synthesis
- 08 End-to-end worked example
- 09 Quick reference: classes & cases

## §01 FTA philosophy & invariants

---

Before walking through mechanics, three principles that drive every design decision in the FTA phase. Each constrains what the agent is allowed to do at every step.

### 1. HYBRID (SPEC + CODE), NOT PURE EITHER

A pure-spec fault tree generates plausible-sounding branches that have no anchor in the actual UE codebase – the agent invents file names, function names, and log strings. A pure-code fault tree has the opposite problem: it finds real code paths but can't relate them to 3GPP-defined procedures, so the resulting "root cause" can't cite a spec violation.

v6's hybrid approach binds them: each top-level branch is a *3GPP-defined phase* (from `spec_query skeleton`) **with** a *UE source module* assigned to it (from `code_search bind_module`). Both anchors are required. If a branch has a spec phase but no module, it's marked `unbindable`; if a phase has no spec basis, it doesn't exist as a branch at all.

### 2. EVIDENCE GATES, NOT REASONING CHAINS

Every claim the FTA pipeline makes is backed by at least one log row or code snippet retrieved during this same pipeline run. No claim is supported by "the agent thinks this is likely." This is enforced through **two gates**:

- **Gate A** (signaling, Phase 3.2) – every top-level branch must either show clean signaling traffic (PRUNE) or signaling failure indicators (CONFIRM) before progressing
- **Gate B** (trace, Phase 3.3) – every failure mode (child branch) must show literal log strings in trace data before becoming a base event

If both gates are silent, the hypothesis is rejected – not "the agent isn't sure." Rejection is the default outcome.

### 3. MECHANICAL SYNTHESIS, NOT CREATIVE INTERPRETATION

The final root cause class (Phase 3.5) is determined by a small decision tree over already-populated state file fields. The agent does not "interpret" the evidence to decide what kind of failure it is – the

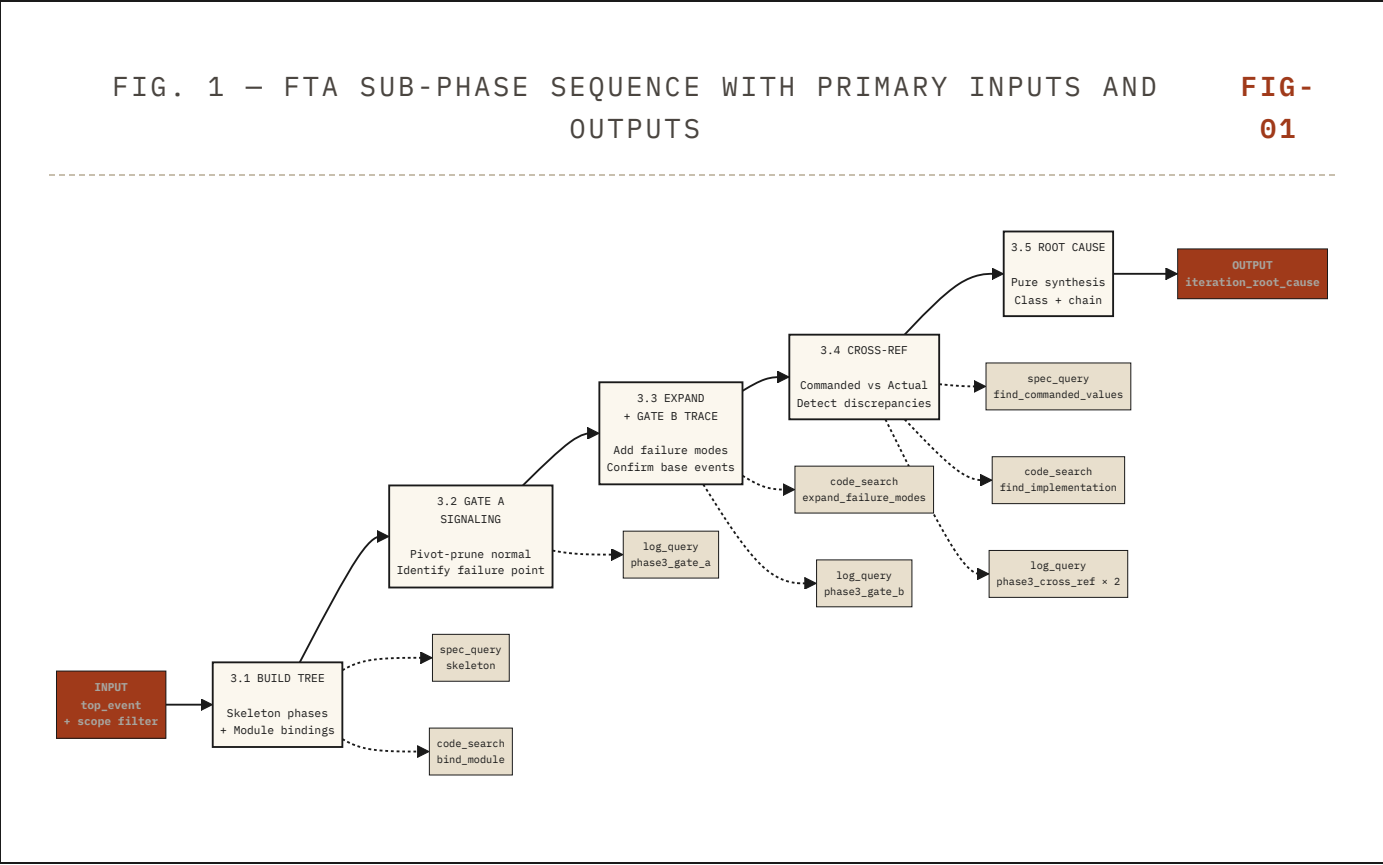
class falls out mechanically from whether cross-reference detected a value discrepancy, whether multiple base events are linked, whether implementation location was identified, and so on.

### **ANTI-HALLUCINATION IS ENFORCED BY THE PYTHON TOOLS, NOT BY SKILL DISCIPLINE**

The wrapper-skill indirection (the recently-discovered weakness) is documentation only. Real enforcement lives in the Python scripts: `log_query.py` refuses table-isolation violations; `code_search.py` extracts log strings only from a fixed set of macros; `spec_query.py` preserves verbatim spec notation. **Every keyword used in any FTA query traces to a prior tool invocation in the same pipeline run** – and the scripts auto-append the audit trail so this is verifiable post-hoc.

## §02 Pipeline at a glance

Five sub-phases run in sequence per iteration. The first four call retrieval tools; Phase 3.5 is pure synthesis (no tool calls). Halts happen if a phase returns insufficient evidence – the iteration controller then surfaces the situation at Checkpoint B.



### SUB-PHASE SUMMARY TABLE

| PHASE | SKILL               | PRIMARY INPUT               | PRIMARY OUTPUT                                        | TOOLS CALLED                                |
|-------|---------------------|-----------------------------|-------------------------------------------------------|---------------------------------------------|
| 3.1   | 3gpp-fta-build-tree | iteration's input_top_event | hybrid_tree.branches[] (top-level only, with modules) | spec_query sk<br>code_search<br>bind_module |
|       |                     |                             |                                                       |                                             |

| PHASE | SKILL                      | PRIMARY INPUT                                        | PRIMARY OUTPUT                                                                                                              | TOOLS CALLED                                                                              |
|-------|----------------------------|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| 3.2   | 3gpp-fta-evaluate-branches | hybrid_tree.branches[]<br>with<br>mandatory_messages | <code>branch[].gate_a_result</code><br><code>pruned_branches[]</code>                                                       | log_query<br>phase3_gate_a<br>spec_query<br>extract_ies (                                 |
| 3.3   | 3gpp-fta-evaluate-branches | surviving branches<br>from 3.2                       | <code>branch[].children[]</code><br>(failure modes)<br><code>base_events[], rejected[],</code><br><code>open_items[]</code> | code_search<br>expand_failur<br>log_query<br>phase3_gate_b                                |
| 3.4   | 3gpp-fta-cross-reference   | confirmed<br>base_events[]                           | <code>cross_reference_findings[]</code><br>with<br><code>commanded/actual/delta</code>                                      | spec_query<br>find_commande<br>code_search<br>find_implemen<br>log_query<br>phase3_cross_ |
| 3.5   | 3gpp-fta-root-cause        | full iteration slice                                 | <code>iteration_root_cause</code><br>(class, description,<br>chain, evidence)                                               | None – pure syn                                                                           |



### §03 Phase 3.1 – Build hybrid tree

---

Goal: produce the *top-level shape* of the fault tree. Children (failure modes) are NOT added here – that happens in 3.3 only on branches that survive Gate A. This phased construction matters: building children for branches that turn out to be normal-passage wastes tool calls and produces noise in the final report.

#### MECHANICS – STEP BY STEP

|   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <p><b>Read the iteration's input top event</b></p> <p>Slice-read <code>fta_iterations[iter-1].input_top_event</code> . For iteration 1, this came from Checkpoint A; for <math>\text{iter} \geq 2</math>, it was set by the iteration controller when the user chose "dig deeper" at the prior Checkpoint B.</p> <p>-----</p> <p><code>READfta_iterations[iter-1].input_top_event</code></p> <p><code>SPEC-ANCHORED?iter 1: usually true · iter <math>\geq</math>2: usually false</code></p>                                                                                                                                                                                                                                                        |
| 2 | <p><b>Query the 3GPP spec skeleton</b></p> <p>Call <code>spec_query.py --operation skeleton</code> with the procedure name, RAT, and the iteration's top event. The GraphRAG returns a list of <b>standardized procedure phases</b> – each with a unique ID, name, spec reference, expected mandatory messages, and protocol layer.</p> <p>For a 5G handover failure: typical skeleton returns ~3 phases (RRC Reconfiguration → Target Cell Sync → Reconfiguration Complete). For a deeper iteration drilling into "Preamble_Power_Error", the skeleton likely returns nothing – that triggers the fallback.</p> <p>-----</p> <p><code>TOOLspec_query.py skeleton</code></p> <p><code>RETURNSpHases[] with mandatory_messages + spec_ref</code></p> |

3

**Detect empty skeleton → fallback path**

If skeleton returns `phases: []`, the input top event isn't a standardized 3GPP procedure – common at iteration  $\geq 2$  when drilling into a code-level cause like "Preamble\_Power\_Error" or "RACH\_Timeout". Set `hybrid_tree.spec_skeleton_returned_empty = true` and call `spec_query.py --operation generate_hypotheses` instead. Each returned hypothesis becomes a top-level branch.

Record the fallback in the iteration record: `hybrid_tree.fallback_used = "generate_hypotheses + code-only"`. The Phase 4 report shows which iterations were spec-anchored versus code-fallback.

---

**TRIGGER**`phases == []`

**EFFECT**`hypotheses[]` used in place of `phases[]`

4

**Bind UE source modules to each branch**

For each phase (or fallback hypothesis), call `code_search.py --operation bind_module` with the phase name, spec ref, and protocol layer. The semantic search returns up to 3 modules with `relevance: HIGH | MEDIUM`, each with file path, primary function, and brief evidence (e.g. "function processes RRCReconfiguration with reconfigurationWithSync IE").

A branch with no module binding becomes `unbindable` – still kept in the tree (Gate A can still evaluate it from signaling alone), but Phase 3.3 expansion will skip it.

---

**TOOL (PER BRANCH)**`code_search.py bind_module`

**EMPTY RESULT**`branch` marked unbindable, kept in tree

**5****Assemble and persist the tree**

Construct the hybrid\_tree JSON: gate\_at\_top = "OR" (default unless spec says AND), each branch carries id, name, spec\_ref, mandatory\_messages[], modules[], status: "unevaluated", children: [], gate\_a\_result: null.

Atomic write to fta\_iterations[iter-1].hybrid\_tree .Phase 3.1 ends; workflow dispatches Phase 3.2.

---

WRITEfta\_iterations[iter-1].hybrid\_tree

NEXTworkflow → 3.2 evaluate-branches

**WORKED EXAMPLE – 5G HO ITERATION 1**



## PHASE 3.1 · ITERATION 1 · SPEC-ANCHORED

**INPUT TOP EVENT: 5G\_HO\_EXECUTION\_FAILURE (RRCRECONFIGURATION RECEIVED, NO FOLLOWING RAR)****Step 2 – spec\_query skeleton returns:**

```
{
  "operation": "skeleton",
  "procedure": "Intra-AMF 5G Handover",
  "spec_refs": ["TS 38.331 §5.5", "TS 23.502 §4.9.1"],
  "gate_at_top": "OR",
  "phases": [
    { "id": "P1", "name": "RRC_Signaling_Phase",
      "spec_ref": "TS 38.331 §5.5.1",
      "mandatory_messages": ["RRCReconfiguration with reconfigurationWithSync"],
      "protocol_layer": "RRC" },
    { "id": "P2", "name": "Target_Cell_Sync_Phase",
      "spec_ref": "TS 38.321 §5.1",
      "mandatory_messages": ["RRCReconfigurationComplete"],
      "protocol_layer": "MAC/PHY" }
  ]
}
```

**Step 4 – code\_search bind\_module for each phase:**

```
P1 → modules: [{ file: "rrc_ho_handler.cpp",
                  primary_function: "handle_rrc_reconfig_with_sync",
                  relevance: "HIGH" }]
P2 → modules: [{ file: "phy_sync_task.c",
                  primary_function: "phy_sync_rach_send",
                  relevance: "HIGH" }]
```

**Resulting hybrid\_tree (Step 5):**

```
fta_iterations[0].hybrid_tree = {
  gate_at_top: "OR",
  spec_skeleton_returned_empty: false,
  fallback_used: null,
  branches: [
    { id: "P1", name: "RRC_Signaling_Phase",
      spec_ref: "TS 38.331 §5.5.1",
      mandatory_messages: ["RRCReconfiguration with reconfigurationWithSync"],
      modules: [{file: "rrc_ho_handler.cpp", primary_function: "handle_rrc_reconfig_wit
```

```
    status: "unevaluated", children: [], gate_a_result: null },  
  { id: "P2", name: "Target_Cell_Sync_Phase",  
    spec_ref: "TS 38.321 §5.1",  
    mandatory_messages: ["RRCReconfigurationComplete"],  
    modules: [{file: "phy_sync_task.c", primary_function: "phy_sync_rach_send"}],  
    status: "unevaluated", children: [], gate_a_result: null }  
]  
}
```

### WHY NO CHILDREN YET?

If we expanded P1's failure modes now and P1 turned out to be a normal passage in Gate A, those failure modes were generated for nothing – wasted tool calls and noise in the report. Phase 3.3 expands only the branches that survive Gate A. This is the central efficiency tradeoff of pivot-pruning: **do the cheap discrimination first, then drill.**

## §04 Phase 3.2 – Gate A pivot-pruning

---

Goal: classify each top-level branch as **normal-passage**, **failure-here**, **absent**, **unclear**, or **N/A** using *signaling data only*. This is "pivot-pruning" because the result is one-way: branches confirmed normal are removed from further analysis, never re-evaluated. Wrong prune = wrong root cause, so the bar for normal\_passage requires positive evidence, not just absence of red flags.

### THE FIVE GATE A CASES

#### CASE A

##### **normal\_passage**

All mandatory messages received cleanly, no failure indicators, expected ordering.

PRUNE

#### CASE B

##### **failure\_here**

Messages present but with failure indicators: REJECT, timer expiry, retransmission limit reached, missing follow-up.

PROCEED TO 3.3

#### CASE C

##### **absent**

Mandatory messages entirely missing from signaling. Could mean PHY layer failure not visible to RRC.

PROCEED TO 3.3

#### CASE D

##### **unclear**

Partial messages, ambiguous ordering, or signaling sparseness – cannot confidently classify either way.

OPEN ITEM

#### CASE N/A

##### **code-only branch**

Iteration  $\geq 2$  fallback hypothesis with no defined mandatory\_messages. Skip Gate A entirely.

→ 3.3 DIRECTLY

### MECHANICS – STEP BY STEP

**1****Prioritize branches with absent-signaling hints**

If iteration 1: sort branches so the ones whose `mandatory_messages` appear in `phase2_ecf.observable_symptoms.missing_events` come first. Phase 2 already noticed which expected messages didn't appear – those branches are the most likely failure points and worth evaluating first. (Iteration  $\geq 2$ : just follow tree order.)

**2****Optional IE refinement**

If `mandatory_messages` is generic (e.g. "RRCReconfiguration" matches many messages), call `spec_query.py --operation extract_ies` to get the specific IE names that distinguish this branch's instance. Pass these as `--ie-names` to the Gate A log query.

---

**TOOL** (OPTIONAL) `spec_query.py extract_ies`

**3****Query signaling log for this branch**

Call `log_query.py --phase-tag phase3_gate_a --table UE_3gpp_signaling_log` with the branch's `mandatory_messages` as keywords, the iteration's time window, and the branch ID as `--hypothesis-id`. Table isolation enforced at script entry – Gate A cannot touch `UE_Trace_log`.

Returns up to 5 key events: timestamp, layer, direction, message name, key IE values (KPIs), cause codes if present.

---

**TOOL** `log_query.py phase3_gate_a`  
**TABLE** `UE_3gpp_signaling_log ONLY`

|   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4 | <p><b>Classify into one of five cases</b></p> <p>The skill maps the <code>key_events</code> shape onto the five cases above. Heuristics:</p> <ul style="list-style-type: none"><li>• <b>Case A normal_passage</b> – all mandatory messages present, no cause codes, expected sequence intact, no timer-expiry markers</li><li>• <b>Case B failure_here</b> – any of: REJECT message, non-zero cause code, timer expiry indicator, max-retx message present</li><li>• <b>Case C absent</b> – <code>keywords_with_hits</code> empty for this branch's mandatory messages</li><li>• <b>Case D unclear</b> – partial hits, ambiguous ordering, or count below expected threshold</li></ul> <p>Write the result to <code>branch.gate_a_result</code> with the verdict and the supporting evidence snippet.</p> |
| 5 | <p><b>Update branch status and lists</b></p> <p>Set <code>branch.status</code> accordingly. Case A → append to <code>pruned_branches[]</code> and skip Phase 3.3. Cases B &amp; C → proceed to Phase 3.3. Case D → append to <code>open_items[]</code> with a note about what's missing, and skip Phase 3.3.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

## THE "ALL BRANCHES NORMAL" HALT

### FAILURE NOT IN SKELETON

If after Gate A every top-level branch is **Case A normal\_passage**, the skill halts with `all_phases_normal_failure_not_in_skeleton`. This means: signaling shows the entire procedure completed without obvious failure, yet the engineer reported a failure. Three explanations are possible:

1. The failure is at a layer below signaling (PHY/MAC/DSP) – needs trace data the FTA pipeline cannot reach because all branches got pruned at signaling
2. The top event was misclassified at Phase 2 – engineer should refine and re-run
3. The signaling capture is incomplete (sparse log coverage)

The iteration controller surfaces this at Checkpoint B with a recommendation depending on which signals fire (typically `abort` with a request for more trace data).

## WORKED EXAMPLE CONTINUED – 5G HO ITERATION 1

## PHASE 3.2 · ITERATION 1

**SORT: MISSING\_EVENTS = ["RRCRECONFIGURATIONCOMPLETE"] → P2  
EVALUATED FIRST**

**Gate A query for P2 (Target\_Cell\_Sync\_Phase):**

```
log_query.py --phase-tag phase3_gate_a \
  --table UE_3gpp_signaling_log \
  --keywords "RRCReconfigurationComplete" \
  --time-window-start-ms 14021000 --time-window-end-ms 14023000 \
  --hypothesis-id P2

→ matched_event_count: 0
  keywords_with_hits: []
  keywords_missed: ["RRCReconfigurationComplete"]
  evidence_summary: "No RRCReconfigurationComplete observed in time window;
                    last upstream message was RRCReconfiguration at 14:02:11.00"

P2.gate_a_result = { result: "absent", evidence: "..." }
P2.status = "absent" → proceed to Phase 3.3
```

**Gate A query for P1 (RRC\_Signaling\_Phase):**

```
log_query.py --phase-tag phase3_gate_a \
  --table UE_3gpp_signaling_log \
  --keywords "RRCReconfiguration with reconfigurationWithSync" \
  --hypothesis-id P1

→ matched_event_count: 1
  keywords_with_hits: ["RRCReconfiguration"]
  key_events: [{ timestamp: "14:02:11.00", layer: "RRC", direction: "DL",
                  message: "RRCReconfiguration",
                  kpis: { reconfigurationWithSync: true } }]
  evidence_summary: "Clean RRCReconfiguration with reconfigurationWithSync received"

P1.gate_a_result = { result: "normal_passage", evidence: "clean RRCReconfig at 14:02:11" }
P1.status = "pruned_normal" → appended to pruned_branches[], SKIP 3.3
```

**After Phase 3.2:** 1 branch (P2) proceeds to Phase 3.3; 1 branch (P1) pruned with positive evidence of normal passage.

## §05 Phase 3.3 – Dynamic expansion + Gate B

---

Goal: for each branch that survived Gate A (Cases B, C, or N/A), discover its specific failure modes from UE source code and validate each against trace logs. This is where the fault tree gains its sub-branches and where individual base events are confirmed.

### THE TWO-STEP PATTERN PER SURVIVING BRANCH

#### STEP A – EXPAND FAILURE MODES (CODE)

Call `code_search.py expand_failure_modes` on the branch's bound module. The script searches for error-handling blocks: timeout/retry counters, validation failures, clamp/saturate logic, exception throws. For each block it extracts:

- A name (e.g. `RACH_Timeout` , `Preamble_Power_Error` )
- Detection code summary ( $\leq 80$  chars)
- **Log keywords – literal strings from MSG\_HIGH/SYS\_ERR/etc. macros only**
- Log tags (thread/component identifiers)
- Function signature

**Fallback:** if empty, call `spec_query expand_sub_causes` to get spec-derived sub-causes instead.

#### STEP B – GATE B TRACE VERIFICATION

For each failure mode just added, call `log_query.py phase3_gate_b` against `UE_Trace_log` with the failure mode's literal `log_keywords` and `log_tags`. Three outcomes:

- CONFIRMED – matched events found → append to `base_events[]`
- REJECTED – zero matches → append to `rejected[]`
- UNCLEAR – partial/ambiguous → append to `open_items[]`

**Anti-laziness rule:** ALL failure modes of a branch are evaluated even if one is already confirmed. Other base events may exist in parallel.



## MECHANICS – STEP BY STEP

|   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <p><b>Loop over surviving branches</b></p> <p>Process each branch with status <code>failure_here</code> , <code>absent</code> , or <code>not_applicable</code> . Pruned branches and open-items are not expanded.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 2 | <p><b>Call <code>code_search.py expand_failure_modes</code></b></p> <p>Per branch, pass the bound module name and the branch's phase context. The script runs three search queries:</p> <ol style="list-style-type: none"> <li>1. "&lt;module&gt; error handling failure paths"</li> <li>2. "&lt;module&gt; timeout retry max limit reached"</li> <li>3. "&lt;module&gt; validation reject invalid"</li> </ol> <p>From the returned snippets, the script extracts log strings <b>only from this fixed set of macros</b>: <code>MSG_HIGH</code>, <code>NAS_MSG_HIGH</code>, <code>LOG_MSG</code>, <code>QCRIL_LOG_DEBUG</code>, <code>RRC_LOG_MSG</code>, <code>SYS_ERR</code>, <code>DS_MSG_HIGH_1</code>, <code>printf("[TAG]...")</code>, <code>fprintf(stderr, ...)</code>, <code>syslog</code>. <b>No inference</b> – if the macro literal isn't there, the <code>log_keyword</code> isn't there.</p> <hr/> <pre>TOOLcode_search.py expand_failure_modes FALLBACK IF EMPTYspec_query.py expand_sub-causes</pre> |
| 3 | <p><b>Append failure modes as branch children</b></p> <p>For each failure mode returned, assign an ID <code>&lt;branch_id&gt;.&lt;n&gt;</code> (e.g. P2.1, P2.2), add to <code>branch.children[]</code> with status: <code>"unevaluated"</code> , <code>gate_b_result: null</code> .</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

|   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4 | <p><b>For each child: call log_query phase3_gate_b</b></p> <p>Query UE_Trace_log with the failure mode's literal log_keywords and log_tags as filters. Time window from scope_filter. Pass child id as --hypothesis-id .</p> <p>Keyword fallback chain (stop at first hit, never invent):</p> <ol style="list-style-type: none"> <li>1. failure_mode.log_keywords from code_search</li> <li>2. failure_mode.function_signature as message LIKE pattern</li> <li>3. extract_ies IE names if available</li> <li>4. STOP – mark Gate B unclear</li> </ol> <hr/> <p>TOOLlog_query.py phase3_gate_b</p> <p>TABLEUE_Trace_log ONLY</p> |
| 5 | <p><b>Classify each child and update lists</b></p> <ul style="list-style-type: none"> <li>• matched_event_count &gt; 0 → status: "base_event_confirmed", append to base_events[]</li> <li>• matched_event_count == 0 → status: "rejected", append to rejected[]</li> <li>• partial/ambiguous → status: "open", append to open_items[]</li> </ul> <p>Continue to next child until ALL children of this branch are evaluated. Then move to next surviving branch.</p>                                                                                                                                                              |

## WORKED EXAMPLE CONTINUED – 5G H0 ITERATION 1

## PHASE 3.3 · ITERATION 1 · ONLY P2 SURVIVING

**EXPAND P2 (TARGET\_CELL\_SYNC\_PHASE, BOUND TO PHY\_SYNC\_TASK.C)****Step 2 – code\_search expand\_failure\_modes returns:**

```
{
  "operation": "expand_failure_modes",
  "module": "phy_sync_task.c",
  "failure_modes": [
    { "id_suffix": "1", "name": "RACH_Timeout",
      "detection_code": "preamble retransmission counter compared to max",
      "log_keywords": ["RACH attempt", "max preamble trans reached"],
      "log_tags": ["THREAD_PHY"],
      "function_signature": "phy_sync_rach_send(...)",
      "layer": "PHY" },
    { "id_suffix": "2", "name": "Preamble_Power_Error",
      "detection_code": "Tx power validation underflow",
      "log_keywords": ["Tx Power clamped", "Underflow limit"],
      "log_tags": ["DSP_CALC"],
      "function_signature": "dsp_calc_preamble_tx_power(...)",
      "layer": "PHY/DSP" }
  ]
}
```

**Step 3 – appended as children:** P2.children = [P2.1 RACH\_Timeout, P2.2 Preamble\_Power\_Error]

**Step 4 – Gate B for P2.1:**

```
log_query.py --phase-tag phase3_gate_b --table UE_Trace_log \
  --keywords "RACH attempt" "max preamble trans reached" \
  --log-tags "THREAD_PHY" \
  --hypothesis-id P2.1

→ matched_event_count: 11
key_events: [
  { timestamp: "14:02:11.50", log_tag: "THREAD_PHY",
    message: "[THREAD_PHY] RACH attempt 10 failed, max preamble trans reached" }
]
evidence_summary: "RACH attempts 1-10 logged; max preamble transmission limit hit"

P2.1.gate_b_result = { result: "confirmed", evidence: "..." }
P2.1.status = "base_event_confirmed" → base_events[] += P2.1
```

**Step 4 – Gate B for P2.2:**

```
log_query.py --phase-tag phase3_gate_b --table UE_Trace_log \
  --keywords "Tx Power clamped" "Underflow limit" \
  --log-tags "DSP_CALC" \
  --hypothesis-id P2.2

→ matched_event_count: 11
  key_events: [
    { timestamp: "14:02:11.45", log_tag: "DSP_CALC",
      message: "[DSP_CALC] Tx Power clamped at -10 dBm (Underflow limit)" }
  ]
  evidence_summary: "DSP underflow clamp triggered on every RACH attempt"

P2.2.gate_b_result = { result: "confirmed", evidence: "..."}
P2.2.status = "base_event_confirmed" → base_events[] += P2.2
```

**After Phase 3.3:** base\_events = [P2.1, P2.2] · rejected = [] · open\_items = []

### WHY BOTH P2.1 AND P2.2 ARE CONFIRMED EVEN THOUGH P2.1 LOOKS LIKE A CONSEQUENCE

Phase 3.3 confirms *presence in trace data*, not causality. P2.1 (RACH timeout) and P2.2 (Preamble power error) both have observable trace evidence – both are base events. Whether one caused the other is a Phase 3.4 question, answered by cross-reference. Confirming both keeps the option of MULTI\_CAUSE if cross-ref can't disambiguate.

## §06 Phase 3.4 – Cross-reference

---

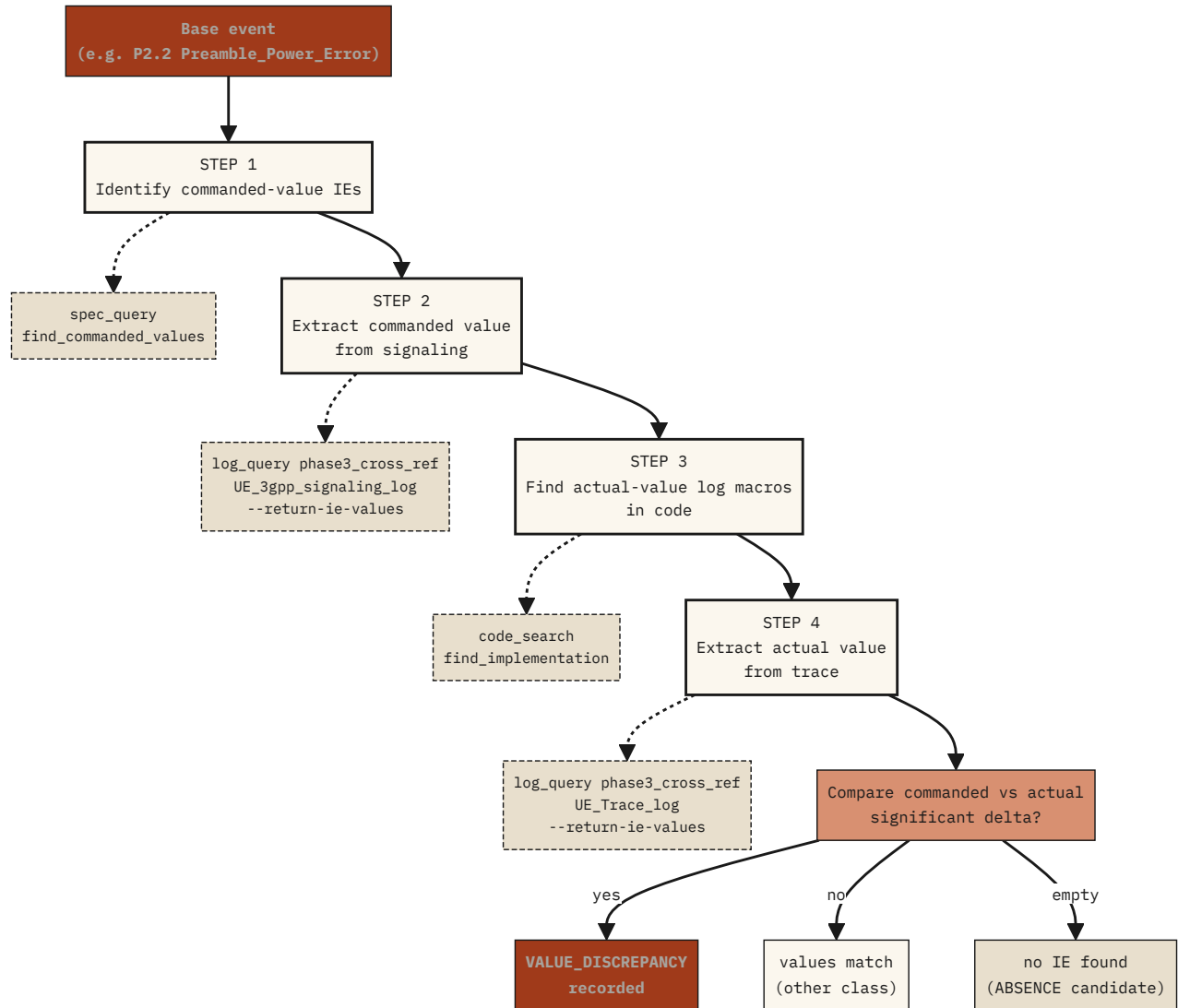
Goal: for each confirmed base event, determine whether the UE applied the value the network commanded. A discrepancy between commanded value (in signaling) and actual value (in trace) is the strongest possible root cause signal – it identifies *exactly which IE* the UE failed to honor, and (via implementation location) *which code function* was responsible.

This is the critical v4-introduced mechanism, preserved verbatim in v6. It's what makes the difference between "RACH failure" (symptom) and "DSP clamps Tx power at -10 dBm despite -110 dBm commanded" (actionable root cause).

### THE FOUR-STEP CROSS-REFERENCE FLOW

FIG. 2 — CROSS-REFERENCE DATA FLOW PER BASE EVENT

FIG-02



## STEP-BY-STEP MECHANICS

**1****Identify commanded-value IEs**

For the base event, collect the upstream signaling messages – typically the `mandatory_messages` of the base event's parent branch (the messages that *set up* the failing operation). Call `spec_query.py find_commanded_values` with the base event name, layer, and upstream messages.

The GraphRAG returns IEs that the network commanded the UE to apply at this layer. Each entry has IE name, source message, semantic meaning, `spec_ref`, expected UE action, and range/unit.

**Empty result is valid.** Many base events have no commanded-value IE – typical of timer-expiry failures or code-internal logic errors. In that case write the finding with `commanded_value_ies: []` and a note; Phase 3.5 will classify as ABSENCE or TIMER\_EXPIRY.

---

```
TOOLspec_query.py find_commanded_values
EMPTY RESULTvalid → ABSENCE candidate
```

**2****Extract commanded value from signaling**

For each commanded IE returned above, query the signaling table with `--return-ie-values true` to extract the actual value the network sent. The script parses the `ie_values` JSON column for the specific IE name and returns `ie_value_extractions[]` with value + units + source message + timestamp.

**If extraction returns nothing** – the IE was in the spec but absent from this run's signaling – append to `open_items[]` with note. DO NOT fabricate the value.

---

```
TOOLlog_query.py phase3_cross_ref (signaling)
--RETURN-IE-VALUEtrue (always for cross-ref)
```

3

**Find actual-value log macros in code**

Call `code_search.py find_implementation` with a description of what the UE was expected to do with the commanded IE (e.g. "DSP calculates and applies transmit power"), the spec IE name as a hint, scope procedure, and scope layer. The script finds code paths where the UE applies this IE and identifies log macros that report the applied value at runtime.

Returns: found (bool), confidence, literal log\_keywords/log\_tags from the macros that print the applied value, function names, implementation file paths, evidence summary.

**If found: false** – no code path that logs this IE's application – mark unverifiable, append to open items.

---

```
TOOLcode_search.py find_implementation
RETURNSlog_keywords + implementation_files
```

4

**Extract actual value from trace**

Call `log_query.py phase3_cross_ref` against `UE_Trace_log` with the literal log\_keywords and log\_tags from Step 3, plus `--return-ie-values true`. The script parses numeric values from matched messages using patterns appropriate to the IE type – `-?\d+(\.\d+)? dBm`, `-?\d+ ms`, etc. – with the expected unit from Step 1.

Returns `ie_value_extractions[]` with the value, source trace line, timestamp, and any inline annotations from the log line (e.g. "Underflow limit").

---

```
TOOLlog_query.py phase3_cross_ref (trace)
SOURCEUE_Trace_log
```



|   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5 | <p><b>Compare and classify</b></p> <p>Compute delta between commanded and actual. Apply significance thresholds per IE type:</p> <ul style="list-style-type: none"> <li>• <b>Power (dBm):</b> <math> \Delta  &gt; 3 \text{ dB} \rightarrow \text{significant}</math></li> <li>• <b>Timer (ms):</b> <math> \Delta  &gt; 10\% \text{ of commanded} \rightarrow \text{significant}</math></li> <li>• <b>Boolean / enum:</b> any mismatch <math>\rightarrow \text{significant}</math></li> <li>• <b>Frequency / numeric ranges:</b> any mismatch outside spec tolerance <math>\rightarrow \text{significant}</math></li> </ul> <p>If significant: <code>root_cause_class = "VALUE_DISCREPANCY"</code> in this finding. Document the threshold used. Otherwise leave class <code>null</code> – Phase 3.5 will assign ABSENCE or another class.</p> |
| 6 | <p><b>Record implementation location and write finding</b></p> <p>Set <code>implementation_location</code> from Step 3's <code>implementation_files[0]</code>. Compose <code>interpretation</code> as 1–2 sentences using only populated values (no creative narrative). Append the full finding to <code>fta_iterations[iter-1].cross_reference_findings[]</code>.</p> <p>Repeat all six steps for every confirmed base event. One finding per base event, even when <code>commanded_value_ies</code> was empty.</p>                                                                                                                                                                                                                                                                                                                         |

## WORKED EXAMPLE CONTINUED – 5G HO ITERATION 1

## PHASE 3.4 · ITERATION 1 · CROSS-REF P2.2 (PREAMBLE\_POWER\_ERROR)

## STEP 1 — SPEC\_QUERY FIND\_COMMANDED\_VALUES

```
{
  "operation": "find_commanded_values",
  "base_event_name": "Preamble_Power_Error",
  "commanded_value_ies": [
    {
      "ie_name": "preambleReceivedTargetPower",
      "message": "RRCReconfiguration",
      "meaning": "Network target receive power UE must transmit at",
      "spec_ref": "TS 38.331 §6.3.2; TS 38.321 §5.1.3",
      "ue_action": "Calculate Tx power = target - pathloss; apply to PRACH",
      "range_unit": "dBm, integer, -202..-60" }
  ]
}
```

## STEP 2 — COMMANDED VALUE EXTRACTION (SIGNALING)

```
log_query.py --phase-tag phase3_cross_ref \
  --table UE_3gpp_signaling_log \
  --keywords "RRCReconfiguration" \
  --ie-names "preambleReceivedTargetPower" \
  --return-ie-values true --hypothesis-id P2.2

→ ie_value_extractions: [
  { ie_name: "preambleReceivedTargetPower",
    value: "-110 dBm",
    source_msg: "RRCReconfiguration",
    timestamp: "14:02:11.00" }
]

commanded_value = { value: "-110 dBm", source: "signaling at 14:02:11.00" }
```

## STEP 3 — FIND\_IMPLEMENTATION (CODE SEARCH)

```
code_search.py --operation find_implementation \
  --hypothesis-cause "DSP calculates and applies transmit power" \
  --spec-ie-names "preambleReceivedTargetPower" \
  --scope-procedure "Intra-AMF 5G Handover" \
  --scope-layer "PHY"

→ found: true, confidence: HIGH,
  log_keywords: ["Tx Power clamped", "Underflow limit"],
  log_tags: ["DSP_CALC"],
```

```
function_names: ["dsp_calc_preamble_tx_power"],
implementation_files: ["dsp_power_calc.c"],
evidence_summary: "Function applies tx power; logs clamped value on underflow"
```

## STEP 4 — ACTUAL VALUE EXTRACTION (TRACE)

```
log_query.py --phase-tag phase3_cross_ref \
  --table UE_Trace_log \
  --keywords "Tx Power clamped" \
  --log-tags "DSP_CALC" \
  --return-ie-values true --hypothesis-id P2.2

→ ie_value_extractions: [
  { ie_name: "Tx Power", value: "-10 dBm",
    source_msg: "DSP_CALC trace",
    timestamp: "14:02:11.50",
    annotations: ["Underflow limit"] }
]
actual_value = { value: "-10 dBm", source: "trace at 14:02:11.50" }
```

## STEP 5 — COMPARE

```
commanded = -110 dBm
actual    = -10 dBm
delta     = 100 dB

Threshold (dBm): |Δ| > 3 dB → significant ✓ ✓ ✓
root_cause_class = "VALUE_DISCREPANCY"
implementation_location = "dsp_power_calc.c::dsp_calc_preamble_tx_power"
```

## RESULTING FINDING

```
cross_reference_findings[0] = {
  base_event_id: "P2.2",
  commanded_ie_lookup: { ie: "preambleReceivedTargetPower", spec_ref: "TS 38.331 §6.3.2" },
  commanded_value: { value: "-110 dBm", source: "signaling RRCReconfiguration at 14:02:" },
  actual_value: { value: "-10 dBm", source: "trace DSP_CALC at 14:02:11.50" },
  delta: "100 dB",
  root_cause_class: "VALUE_DISCREPANCY",
  implementation_location: "dsp_power_calc.c::dsp_calc_preamble_tx_power",
  interpretation: "Network commanded preambleReceivedTargetPower of -110 dBm but"
```

UE DSP clamped Tx power at -10 dBm due to underflow handling"

}

P2.1 (RACH\_Timeout) is also processed in Phase 3.4, but `find_commanded_values` returns an empty list because RACH timeout itself doesn't have a directly-commanded IE – the spec says "if max preamble transmissions reached, declare failure" without commanding a specific value. So P2.1's finding has empty `commanded_value_ies` and no class set; in Phase 3.5 it'll be marked as a *symptom* of P2.2 (the VALUE\_DISCREPANCY).



## §07 Phase 3.5 – Root cause synthesis

---

Goal: assemble the iteration-local root cause from all prior Phase 3 findings. **Pure synthesis – no tool calls, no new keywords.** The skill reads the iteration's state slice and combines existing fields into the `iteration_root_cause` block. If a field can't be filled from prior phases, it stays `null` – never invented.

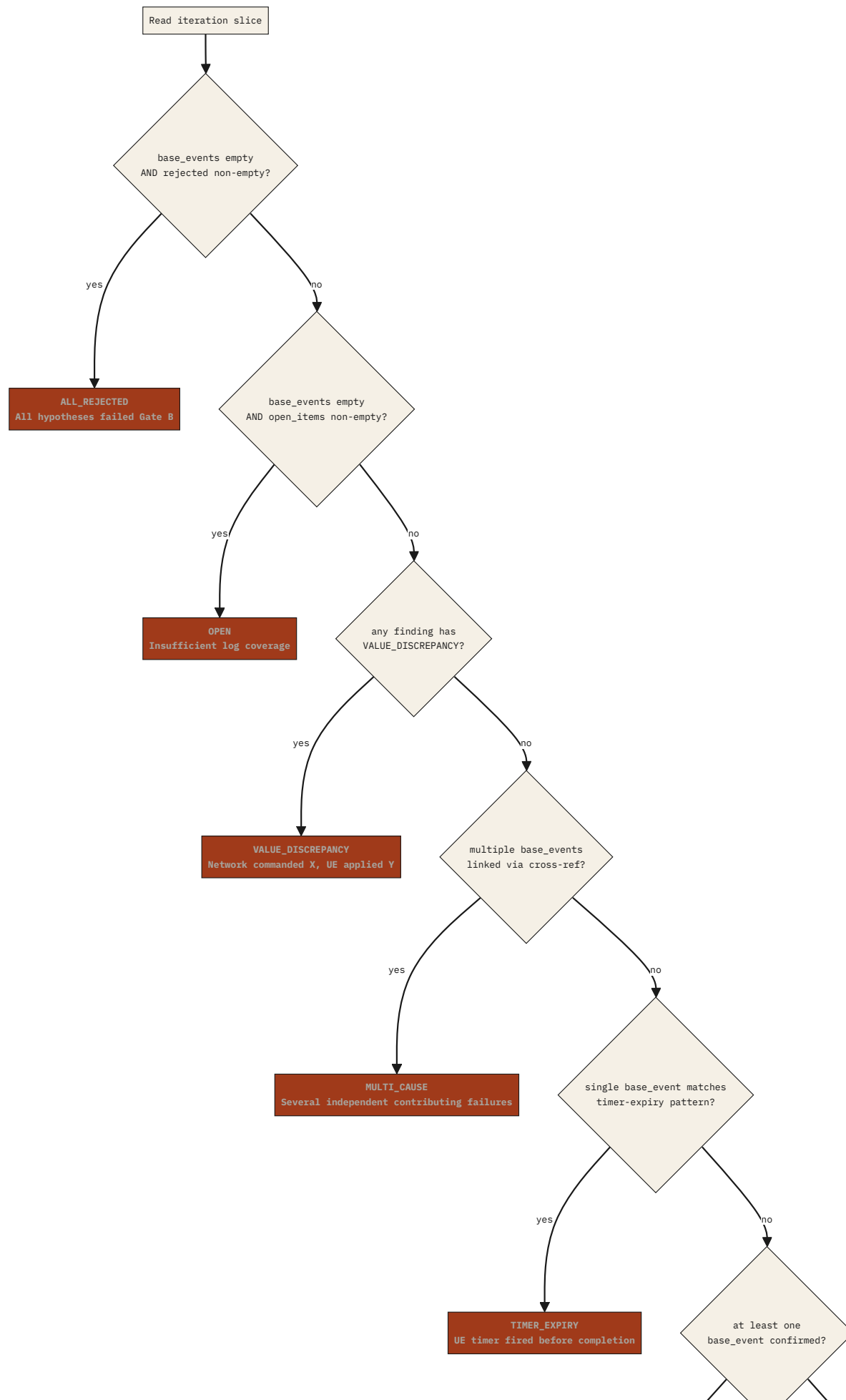
This is also where the **root\_cause\_class** is determined mechanically. One of six classes wins based on a small decision tree over the cross-reference findings and base event sets.

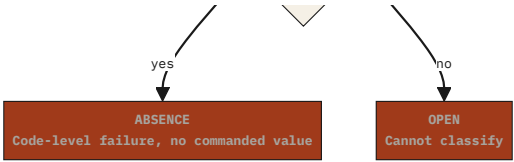
### ROOT CAUSE CLASS – DECISION TREE

FIG. 3 — ROOT CAUSE CLASS DETERMINATION (MECHANICAL, FIRST  
MATCH WINS)

---

**FIG -  
03**





THE SIX ROOT CAUSE CLASSES

| CLASS             | MEANING                                                                                                                            | TYPICAL EVIDENCE PATTERN                                                                                        | CONTROLLER'S NEXT-ITERATION ACTION                                                            |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| VALUE_DISCREPANCY | Network commanded a specific value via an IE; UE actually applied a different value. The strongest, most actionable class.         | Cross-ref finding has commanded_value, actual_value, and significant delta. Implementation location identified. | Recommend <i>dig_deeper</i> into the cause-side base event (the one carrying the discrepancy) |
| ABSENCE           | Confirmed base event exists in trace but no commanded-value IE applies. Typically a code-internal logic error or missing handling. | ≥1 base_event confirmed; cross-ref returned empty commanded_value_ies .                                         | Iter 1: <i>dig_deeper</i> ; iter ≥2: <i>accept_terminal</i>                                   |
| TIMER_EXPIRY      | Single base event is a timer firing. UE waited for something that didn't arrive.                                                   | One base_event matches timer naming pattern (T3*, T3xx, timeout, wait_for_*).                                   | With implementation_location: <i>accept_terminal</i> ; without: <i>dig_deeper</i>             |



| CLASS               | MEANING                                                                                               | TYPICAL EVIDENCE PATTERN                                                                     | CONTROLLER'S NEXT-ITERATION ACTION                                    |
|---------------------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <b>MULTI_CAUSE</b>  | Several base events are confirmed; cross-ref links them but doesn't isolate one as the primary cause. | $\geq 2$ base_events; cross-ref findings show interaction but no clear dominant discrepancy. | Recommend <i>dig_deeper</i> into the highest-confidence base event    |
| <b>ALL_REJECTED</b> | All proposed failure modes failed Gate B – no trace evidence supports any of them.                    | base_events empty; rejected[] non-empty.                                                     | Recommend <i>abort</i> ; Phase 2 may have misidentified the top event |
| <b>OPEN</b>         | Insufficient evidence to confirm or reject – typically sparse log coverage.                           | base_events empty; open_items[] non-empty.                                                   | Recommend <i>abort</i> ; need more trace capture and re-run           |

## SYNTHESIS STEPS

|          |                                                                                                                                                                                                                           |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1</b> | <p><b>Determine root_cause_class via decision tree</b></p> <p>Apply the decision tree in Fig. 3 strictly in order – first match wins. Pure boolean logic over already-populated state slice; no fuzzy interpretation.</p> |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2 | <p><b>Identify failing_phase</b></p> <p>From <code>hybrid_tree.branches[]</code> , the branch with <code>status == "failure_here"</code> (or deepest "absent" branch if no <code>failure_here</code> exists). Record its name in <code>failing_phase</code> ; record names of all pruned branches in <code>pruned_phases[]</code> .</p>                                                                                                                                                                                                                                                                                                                                       |
| 3 | <p><b>Build base_event_chain with cause/symptom relationships</b></p> <p>Iterate <code>base_events[]</code> . For each, check <code>cross_reference_findings</code> : if this base event has a <code>VALUE_DISCREPANCY</code> finding, label its <code>relationship</code> as <code>cause_of_&lt;other_id&gt;</code> where the other is also a base event without its own discrepancy. The non-discrepancy base events are labeled <code>symptom</code> .</p> <p>This is how the chain "P2.2 caused P2.1" emerges – P2.2 has the discrepancy, P2.1 has trace evidence but no commanded value, so P2.1 is labeled symptom and P2.2 is labeled <code>cause_of_P2.1</code>.</p>  |
| 4 | <p><b>Compose description from class-specific template</b></p> <p>One template per class. For <code>VALUE_DISCREPANCY</code>:</p> <div data-bbox="293 1262 1451 1528" style="border: 1px solid black; padding: 10px; margin: 10px 0;"> <p>"Network commanded {commanded_ie} = {commanded_value} via {source_msg} at {source_ts}, but UE applied {actual_value} (observed in trace at {trace_ts} via {implementation_function}). Delta of {delta} exceeds {threshold}. This caused {symptom_base_event_name}, which propagated to top event {input_top_event}."</p> </div> <p>All placeholders fill from already-populated state fields. No invented prose. 2-4 sentences.</p> |

|   |                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5 | <p><b>Pick <code>implementation_location</code> and <code>spec_violation</code></b></p> <p>Both come from the cross-reference finding if <code>VALUE_DISCREPANCY</code>. For other classes: <code>implementation_location</code> from the deepest base event's <code>function_signature</code> ; <code>spec_violation</code> from the base event's <code>spec_ref</code> . Either field may be <code>null</code> if not derivable – never invented.</p> |
| 6 | <p><b>Assemble <code>evidence_chain</code></b></p> <p>Up to 15 entries, in order: iteration context → pruning evidence per pruned branch → Gate A evidence for failing branch → Gate B evidence per confirmed base event → cross-reference values per finding → implementation location reference. Each entry is one line citing the source (signaling row, trace row, code snippet, or <code>spec_ref</code>).</p>                                     |
| 7 | <p><b>Atomic write</b></p> <p>Write the complete <code>iteration_root_cause</code> block to <code>fta_iterations[iter-1]</code> . Do NOT modify <code>meta.current_phase</code> – the iteration controller will pick up next.</p>                                                                                                                                                                                                                       |

## WORKED EXAMPLE CONTINUED – 5G H0 ITERATION 1

## PHASE 3.5 · ITERATION 1 · SYNTHESIS

### STEP 1 — CLASS DETERMINATION

base\_events = [P2.1, P2.2] (non-empty) · cross\_reference\_findings has a  
VALUE\_DISCREPANCY → **class = VALUE\_DISCREPANCY**

### STEP 2 — FAILING\_PHASE

P2 has status `absent` (Gate A found no RRCReconfigurationComplete) → failing\_phase =  
"Target\_Cell\_Sync\_Phase"

pruned\_phases = ["RRC\_Signaling\_Phase"] (from P1 pruned\_normal)

### STEP 3 — BASE\_EVENT\_CHAIN

```
base_event_chain = [
  { id: "P2.1", name: "RACH_Timeout",
    relationship: "symptom" },
  { id: "P2.2", name: "Preamble_Power_Error",
    relationship: "cause_of_P2.1" }
]
```

P2.2 has VALUE\_DISCREPANCY finding → labeled cause. P2.1 has no commanded-value  
IE → labeled symptom.

### STEPS 4-6 — FULL ITERATION\_ROOT\_CAUSE

```
fta_iterations[0].iteration_root_cause = {
  deduced_at: "2026-05-20T03:21:14Z",
  iteration_id: 1,
  input_top_event: "5G_HO_Execution_Failure",
  failing_phase: "Target_Cell_Sync_Phase",
  pruned_phases: ["RRC_Signaling_Phase"],
  base_event_chain: [
    { id: "P2.1", name: "RACH_Timeout", relationship: "symptom" },
    { id: "P2.2", name: "Preamble_Power_Error", relationship: "cause_of_P2.1" }
  ],
  root_cause_class: "VALUE_DISCREPANCY",
  description: "Network commanded preambleReceivedTargetPower = -110 dBm via
    RRCReconfiguration at 14:02:11.00, but UE applied Tx Power = -10 dBm
    (observed in trace at 14:02:11.50 via dsp_calc_preamble_tx_power).
    Delta of 100 dB exceeds 3 dB threshold. This caused
    RACH_Timeout (P2.1), which propagated to top event 5G_HO_Execution_Fail
    commanded_value: "-110 dBm",
```

```

actual_value: "-10 dBm",
implementation_location: "dsp_power_calc.c::dsp_calc_preamble_tx_power",
spec_violation: "TS 38.321 §5.1.3 – UE shall transmit at commanded preamble power",
evidence_chain: [
  "(scope) discriminator: handover fails on 5G NR Band n78, never on LTE",
  "(Phase 2) missing event: RRCReconfigurationComplete after RRCReconfiguration",
  "(Phase 3.2) P1 pruned_normal: clean RRCReconfiguration at 14:02:11.00",
  "(Phase 3.3) P2.1 confirmed: [THREAD_PHY] RACH attempt 10 failed, max preamble tran
  "(Phase 3.3) P2.2 confirmed: [DSP_CALC] Tx Power clamped at -10 dBm (Underflow limi
  "(Phase 3.4) Commanded preambleReceivedTargetPower = -110 dBm",
  "(Phase 3.4) Actual Tx Power = -10 dBm (delta 100 dB)",
  "(Phase 3.4) Implementation: dsp_power_calc.c::dsp_calc_preamble_tx_power"
]
}

```

**Phase 3 complete for this iteration.** Workflow next dispatches the iteration controller, which derives `agent_recommendation: dig_deeper P2.2 (rationale: VALUE_DISCREPANCY at P2.2 → drill into code-level mechanism).`

### WHY THIS ISN'T THE FINAL ROOT CAUSE YET

P2.2's VALUE\_DISCREPANCY tells us *that* the DSP applied -10 dBm instead of -110 dBm. It doesn't yet tell us *why* – what's the bug in `dsp_calc_preamble_tx_power` that produces the clamp? That's iteration 2's job. The user is asked at Checkpoint B whether to dig deeper into P2.2 (run iteration 2 with "Preamble\_Power\_Error" as the new top event) or accept this as terminal. **The choice belongs to the user, not to the agent** – this is the v6 core change from v5.

## §08 End-to-end worked example

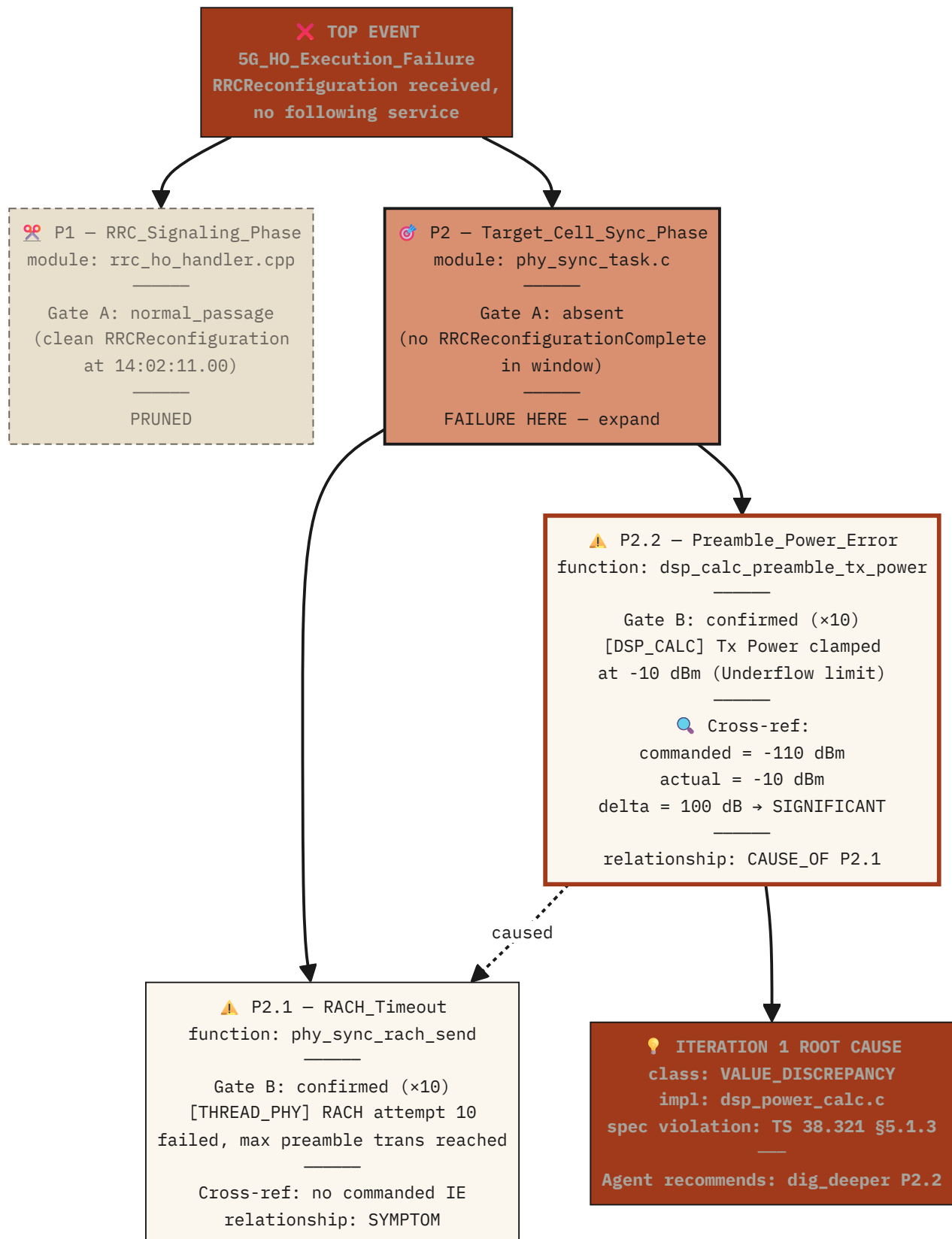
---

Visualization of the entire iteration 1 fault tree after all phases complete, including the iteration 2 chain when the user chose dig\_deeper.

### ITERATION 1 – FULL FAULT TREE VISUALIZATION

FIG. 4 – ITERATION 1 HYBRID TREE WITH ALL PHASE RESULTS

FIG-04

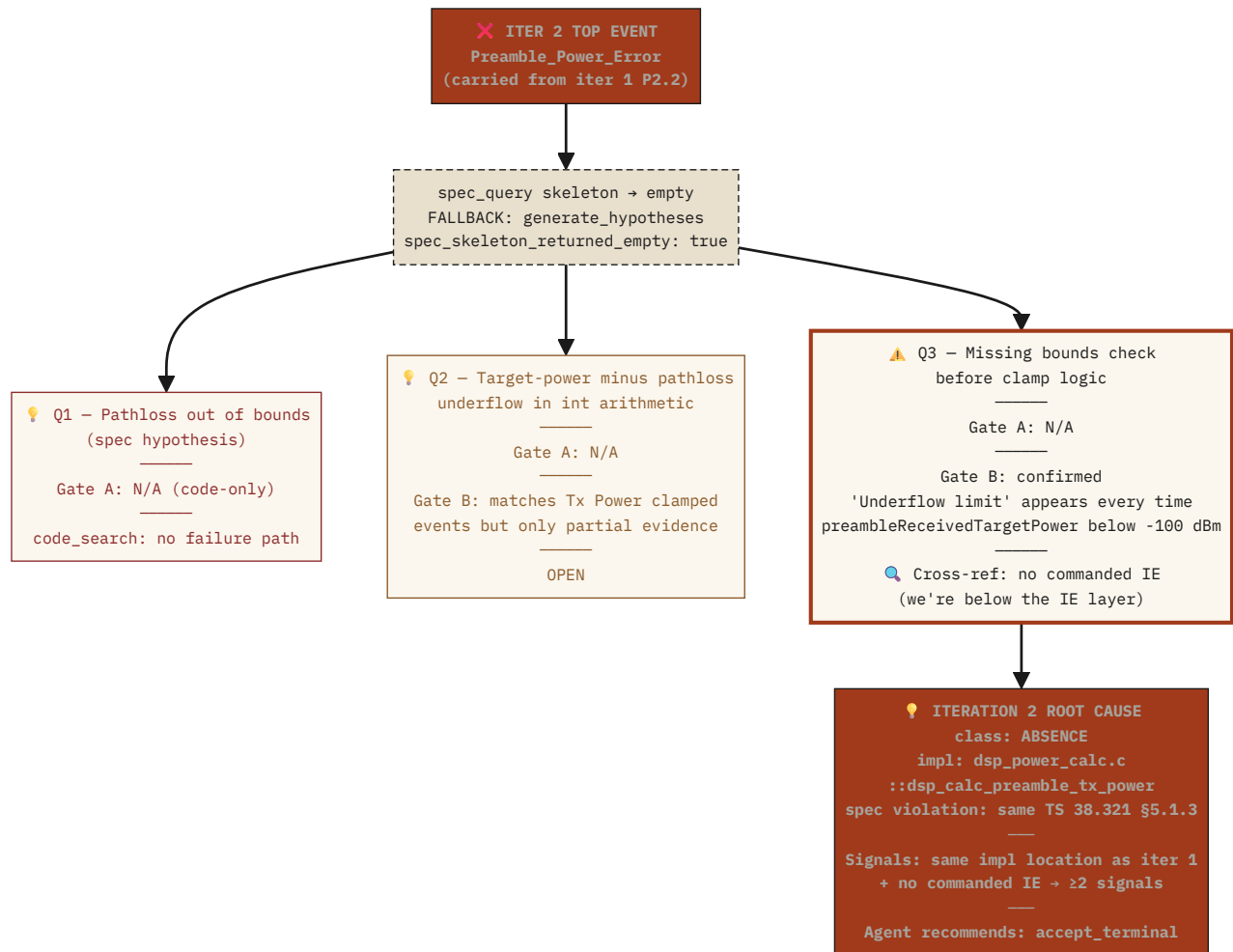


## ITERATION 2 – DRILLING INTO P2.2 (AFTER USER CHOSE DIG\_DEEPER)

The user chose dig deeper into P2.2 at Checkpoint B-1. Iteration 2 begins with `Preamble_Power_Error` as the input top event. The spec skeleton returns empty (no 3GPP procedure for "DSP power calculation"), triggering the `generate_hypotheses` fallback. The fault tree for iteration 2 is code-only:

FIG. 5 – ITERATION 2 HYBRID TREE (CODE-ONLY FALLBACK)

FIG-05



## FINAL CAUSAL CHAIN (AFTER USER ACCEPTS TERMINAL AT CHECKPOINT B-2)

When the user accepts iteration 2 as terminal, the iteration controller synthesizes `phase3_root_cause_chain` across both iterations. The Phase 4 report renders it:



## FINAL CAUSAL CHAIN

---

ITERATION 1: Top Event = 5G\_HO\_Execution\_Failure  
 → cause: VALUE\_DISCREPANCY at P2.2 (Preamble\_Power\_Error)  
 → user chose: dig deeper

ITERATION 2: Top Event = Preamble\_Power\_Error (was P2.2 in iter 1)  
 → cause: ABSENCE at Q3 (Missing\_Bounds\_Check)  
 → user chose: accept terminal

### Terminal root cause:

ABSENCE – dsp\_power\_calc.c::dsp\_calc\_preamble\_tx\_power does not validate input bounds before applying clamp logic. When the network-commanded preambleReceivedTargetPower is below the function's internal underflow threshold, the function silently clamps the computed Tx power to -10 dBm and proceeds without raising an error to the caller. This violates TS 38.321 §5.1.3, which requires the UE to transmit at the commanded preamble power.

### Chain (top-event → terminal):

5G\_HO\_Execution\_Failure (iter 1 top event)  
 ← RACH\_Timeout (P2.1) (symptom in iter 1)  
 ← Preamble\_Power\_Error (P2.2) (cause of P2.1, iter 2 top event)  
 ← Missing\_Bounds\_Check (Q3) (terminal cause)

### User decision audit:

Checkpoint A: confirmed primary (rank 1) [override\_rec: false]  
 Checkpoint B-1: dig\_deeper P2.2 (agent recommended same) [override\_rec: false]  
 Checkpoint B-2: accept\_terminal (agent recommended same) [override\_rec: false]

high\_disagreement\_run: false (0 of 3 decisions were overrides)

## §09 Quick reference – classes & cases

### GATE A OUTCOMES (PHASE 3.2)

| CASE                | TRIGGER                                                            | BRANCH STATUS  | NEXT ACTION | LIST IT LANDS IN         |
|---------------------|--------------------------------------------------------------------|----------------|-------------|--------------------------|
| A<br>NORMAL_PASSAGE | All mandatory messages clean, no failure indicators                | pruned_normal  | SKIP 3.3    | pruned_branches[]        |
| B FAILURE_HERE      | Failure indicators in signaling (REJECT, cause code, timer expiry) | failure_here   | PROCEED 3.3 | (expanded into children) |
| C ABSENT            | Mandatory messages entirely missing                                | absent         | PROCEED 3.3 | (expanded into children) |
| D UNCLEAR           | Partial / ambiguous evidence                                       | open           | SKIP 3.3    | open_items[]             |
| N/A CODE-ONLY       | Iter $\geq 2$ hypothesis with no defined signaling messages        | not_applicable | PROCEED 3.3 | (expanded into children) |

### GATE B OUTCOMES (PHASE 3.3)

| OUTCOME   | TRIGGER                                                             | CHILD STATUS         | LIST IT LANDS IN |
|-----------|---------------------------------------------------------------------|----------------------|------------------|
| CONFIRMED | $\geq 1$ matched_event in trace for the failure mode's log_keywords | base_event_confirmed | base_events[]    |
| REJECTED  | Zero matches in trace; keyword fallback chain also exhausted        | rejected             | rejected[]       |
| UNCLEAR   | Partial / ambiguous; not enough signal to confirm or reject         | open                 | open_items[]     |

## CROSS-REFERENCE OUTCOMES (PHASE 3.4)

| FINDING STATE    | CONDITION                                                                 | ROOT_CAUSE_CLASS FIELD | AFFECTS 3.5 CLASS       |
|------------------|---------------------------------------------------------------------------|------------------------|-------------------------|
| DISCREPANCY      | commanded value present, actual value present, $ \Delta $ above threshold | "VALUE_DISCREPANCY"    | → VALUE_DISCREPANCY     |
| matched          | commanded $\approx$ actual within tolerance                               | null                   | may go to ABSENCE       |
| empty commanded  | find_commanded_values returned []                                         | null (with note)       | ABSENCE or TIMER_EXPIRY |
| extraction empty | IE in spec but absent from this run's signaling/trace                     | null                   | moves to open_items     |

## SIGNIFICANCE THRESHOLDS (PHASE 3.4 DELTA)

| IE TYPE             | THRESHOLD FOR "SIGNIFICANT DELTA"      | EXAMPLE                                                     |
|---------------------|----------------------------------------|-------------------------------------------------------------|
| Power (dBm)         | $ \Delta  > 3 \text{ dB}$              | -110 vs -10 → 100 dB → significant                          |
| Timer (ms)          | $ \Delta  > 10\% \text{ of commanded}$ | commanded 5000ms, actual 6000ms → 20% → significant         |
| Boolean / enum      | Any mismatch                           | commanded "supported", actual "not_supported" → significant |
| Frequency / channel | Outside spec tolerance window          | commanded EARFCN 1800, actual 1900 → significant            |
| Counter / sequence  | Exact mismatch                         | commanded SN 42, actual SN 41 → significant                 |

## HALT CONDITIONS ACROSS THE FTA PHASE

| CONDITION                                    | DETECTED BY           | EFFECT ON ITERATION                                     | CONTROLLER'S RECOMMENDATION                           |
|----------------------------------------------|-----------------------|---------------------------------------------------------|-------------------------------------------------------|
| Spec skeleton empty                          | 3.1 build-tree        | Fallback to generate_hypotheses; tree becomes code-only | (no halt; iteration continues)                        |
| All branches normal-passage (Gate A)         | 3.2 evaluate-branches | HALT – failure not in the skeleton                      | abort (request more trace or refine top event)        |
| code_search<br>expand_failure_modes<br>empty | 3.3 evaluate-branches | Fallback to spec_query<br>expand_sub_causes             | (no halt; iteration continues)                        |
| All failure modes Gate B rejected            | 3.3 + 3.5 combined    | iteration_root_cause class = ALL_REJECTED               | abort (hypotheses don't match this failure)           |
| find_commanded_values<br>empty               | 3.4 cross-reference   | Cross-ref finding has empty<br>commanded_value_ies      | likely ABSENCE class; recommend depends on iter depth |

| CONDITION                                           | DETECTED BY      | EFFECT ON ITERATION                         | CONTROLLER'S RECOMMENDATION               |
|-----------------------------------------------------|------------------|---------------------------------------------|-------------------------------------------|
| Implementation found across same file as prior iter | 3.4 + controller | Termination signal for iteration controller | accept_terminal (code primitive reached)  |
| Iteration budget exhausted                          | controller       | (no FTA effect)                             | accept_terminal (user can still override) |